

Reflexión Individual sobre los Principios SOLID

Entrega individual — Brightspace

Estudiante	Joao Conde (QA Engineer)
Categoría de grupo	Semana 1 - Autoinscripcion
Nombre de grupo	S1 1 - Martes
Repositorio	https://github.com/Lyriom/calidad32

1. ¿Cuál fue el principio más desafiante de aplicar? ¿Por qué?

El que más me costó fue el **Principio de Sustitución de Liskov (LSP)**. La dificultad no estuvo en la teoría, sino en aceptar que heredar no es 'reaprovechar código' sino comprometerse con un contrato de comportamiento. Lo terminé de entender con nuestras figuras: el `Main` recorre una lista de `Figura` y suma todas las áreas sin saber qué subtipo tiene en la mano. Si un solo subtipo —por ejemplo un cuadrado mal modelado— devolviera un valor incoherente o lanzara una excepción, no rompería solo a esa figura, sino a todo el cálculo que confía en el tipo base. Decidir qué métodos iban en `Figura` y cuáles no fue la parte que más discutimos.

2. ¿Cómo crees que SOLID mejora el diseño de software?

Desde el lado de calidad, que es lo que más me interesa, SOLID mejora el diseño porque elimina sorpresas. Un código que respeta estos principios se comporta como uno espera, sin efectos colaterales raros que aparecen recién en producción. Eso hace que las pruebas sean confiables y que, cuando algo falla, sea rápido aislar la clase responsable, porque cada una tiene un rol claro.

3. ¿Qué principio SOLID aplicarías en futuros o actuales proyectos?

Aplicaría **LSP** cada vez que diseñe una jerarquía: antes de que una clase herede de otra, me preguntaría si de verdad puede cumplir todo lo que promete la base, y escribiría pruebas que verifiquen esa sustituibilidad de forma explícita. Junto con LSP me llevo **ISP**, para no forzar a una clase a depender de métodos que no usa —algo típico cuando hay muchos roles o permisos de usuario—.

